

ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
Faculty of Electrical and Computer Engineering
Computer Science Engineering

PRACTICE REPORT

Continuous Integration (CI/CD)

Project: CaféLibro — Library Loan Manager

Course: Software Engineering II

Term: PAO-I - 2026

Team members:

Full name
Darwin Javier Pacheco Paredes
Jonathan Paul Zambrano Ariaga
Freddy Ricardo Asanza Romero
Luis Abrahan Cedeño Arteaga
Giovanni Andre Sambonino Pincay

Professor: Ing. Carlos Mera
Guayaquil – Ecuador

Contents

Contents	2
1. Introduction	3
2. Development	3
2.1 Repository and feature list	3
2.2 Continuous Integration workflow (.github/workflows/ci.yml)	3
2.3 Main branch protection	3
2.4 Parallel development: pull requests per member	3
2.5 The CI blocks a change that breaks the tests	4
2.6 Automating the release (Activity 6)	4
2.7 Decision on Continuous Deployment (Activity 7)	4
3. Questions	4
3.1 Why is CI worth the effort on a team project? What goes wrong without it?	4
3.2 Comparison with at least two other CI/CD tools (Jenkins, GitLab CI, CircleCI, Travis CI, etc.)	4
3.3 Which one would you choose for the course project, and why?	5
3.4 How does CI fit with the way agile teams work in short iterations?	5
4. Conclusions and recommendations	5
5. References	5

1. Introduction

Continuous integration is a development practice in which every member of a team merges their work into a shared repository frequently — at least once a day — and each merge triggers an automated build that compiles the code and runs the full test suite. The core problem it solves is late integration: when developers work in isolation for days or weeks and only combine their changes at the end, conflicts pile up, bugs become hard to locate, and fixing them costs far more time than if they had been caught immediately. By keeping the integration cycle short and automating the verification, the team always knows whether the shared codebase is in a working state.

The goal of this practice was to build a working CI pipeline around CaféLibro using GitHub Actions, enforce branch protection rules so that only verified code can reach the main branch, and reach the boundary with continuous delivery by automating a packaged release on every passing merge. Each team member owned one feature, worked on a dedicated branch, and merged through a reviewed pull request — demonstrating in practice how CI disciplines the collaboration flow on a shared repository.

CaféLibro is a command-line library loan manager for a small library that previously tracked its operations on paper. The program allows staff to register members, lend books, process returns, list a member's active loans, and flag overdue items. All state — books, members, and active loans — is persisted in a JSON file so that separate commands share the same data between runs. The system enforces two basic business rules: a book can only be on loan to one member at a time, and a member may hold at most three books simultaneously.

2. Development

2.1 Repository and feature list

Repository URL: [https://github.com/\[user\]/\[repo\]](https://github.com/[user]/[repo])

Feature assignment (one per member):

#	Feature	Owner	Branch
1	Register a member (name and unique identifier)	Luis Abrahan Cedeño Arteaga	luis_cedeno_feature1
2	Loan a book to a member (a book already on loan cannot be loaned again)	Jonathan Paul Zambrano Ariaga	jonathan_zambrano_feature2
3	Return a book (mark it available again)	Freddy Ricardo Asanza Romero	ricardo_asanza_feature3
4	List the books a member currently has on loan	Jorge Bravo	jorge_bravo_feature4
5	Report overdue loans (given a due date for each loan)	Darwin Javier Pacheco Paredes	darwin_pacheco_feature5

2.2 Continuous Integration workflow (.github/workflows/ci.yml)

The workflow file used is shown below:

This workflow will install Python dependencies, run tests and lint with a single version of Python

For more information see:

<https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python>

name: Python application

on:

push:

branches: ["main"]

pull_request:

branches: ["main"]

permissions:

contents: read

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Set up Python 3.10

uses: actions/setup-python@v3

with:

python-version: "3.10"

- name: Install dependencies

run: |

python -m pip install --upgrade pip

pip install flake8 pytest

if [-f requirements.txt]; then pip install -r requirements.txt; fi

- name: Lint with flake8

run: |

stop the build if there are Python syntax errors or undefined names

flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics

exit-zero treats all errors as warnings. The GitHub editor is 127 chars wide

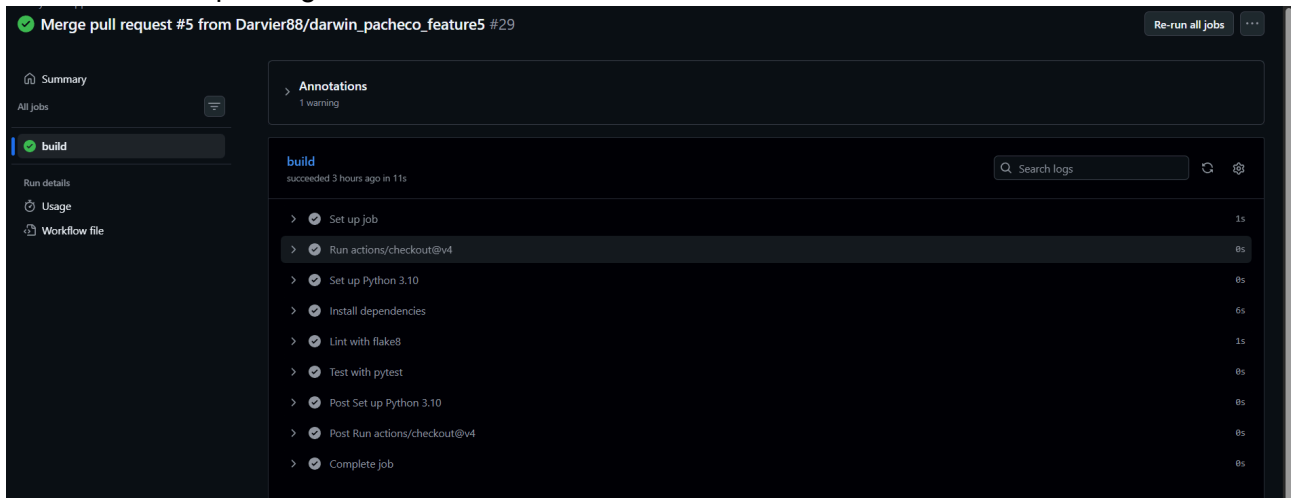
flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127 --statistics

- name: Test with pytest

run: |

pytest

Screenshot of a passing run on the Actions tab:



2.3 Main branch protection

Configuration applied in Settings → Rules/Branches: the CI check must pass and at least one approving review is required before a pull request can be merged into main.

The screenshot shows the GitHub Rulesets configuration page for a ruleset named 'protect-main', which is currently 'Active'. The 'Ruleset Name' field is set to 'protect-main'. Under 'Enforcement status', it is set to 'Active'. The configuration includes several rules:

- ☒ **Restrict deletions**
Only allow users with bypass permissions to delete matching refs.
- ☐ **Require linear history**
Prevent merge commits from being pushed to matching refs.
- ☐ **Require deployments to succeed**
Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.
- ☐ **Require signed commits**
Commits pushed to matching refs must have verified signatures.
- ☒ **Require a pull request before merging**
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
[Show additional settings](#)
- ☒ **Require status checks to pass**
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
[Show additional settings](#)
- ☒ **Block force pushes**
Prevent users with push access from force pushing to refs.

2.4 Parallel development: pull requests per member

Each member built their feature on their own branch, opened a pull request to main (where CI ran automatically), and reviewed another teammate's PR in a round.

Pull Request captures (CI check running on each PR)

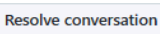
Ricardo Asanza

 feature: Devuelve un libro para que vuelva a estar disponible. #4

Ricardo24A wants to merge 1 commit into `main` from `ricardo_asanza_feature3` 

   Copilot uses AI. Check for mistakes.

 Reply...



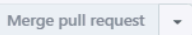


 **Review required**
At least 1 approving review is required by reviewers with write access.

 **All checks have passed**
1 successful check

  Python application / build (pull_request) Successful in 11s Required ...

 **Merging is blocked**
At least 1 approving review is required by reviewers with write access.

 Merge pull request ... You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Darwin Pacheco

 Feature 5 completed #5

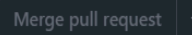
Darvier88 wants to merge 4 commits into `main` from `darwin_pacheco_feature5` 



 **Review required**
At least 1 approving review is required by reviewers with write access.

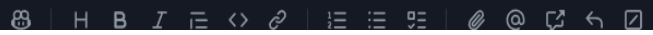
 **All checks have passed**
1 successful check

 **Merging is blocked**
At least 1 approving review is required by reviewers with write access.

 Merge pull request ... You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

 Add a comment

Write Preview 

Jorge Bravo

feat: Listar los libros de un miembro que ha prestado #3

Awaiting approval

Code

Reload2704 wants to merge 4 commits into `main` from `jorge_bravo_feature4`

Conversation 0 Commits 4 Checks 1 Files changed 3

+123



Reload2704 commented 17 minutes ago

Collaborator

Codigo fuente del feature 4.



Reload2704 added 4 commits 20 minutes ago

feat: Listar los libros de un miembro que ha prestado

855bba7

test: Listar los libros de un miembro que ha prestado

c55f3ec

fix: quitar archivo erroneo

f7a4531

fix: renombrar archivo de pruebas para que pytest lo detecte

90086e3



Review required

At least 1 approving review is required by reviewers with write access.

All checks have passed

1 successful check

Merging is blocked

At least 1 approving review is required by reviewers with write access.

Reviewers

Suggestions

Darvier88

Request

At least 1 approving review is required to merge this pull request.

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Unsubscribe

Giovanni Sambonino

feat: Registrar un libro en el catálogo, con un título y un código único, con validaciones y pruebas #2

Awaiting approval

Code

GiovanniSambonino wants to merge 2 commits into `main` from `giovanni_sambonino_feature6`

Conversation 0 Commits 2 Checks 1 Files changed 5

+94



GiovanniSambonino commented 16 minutes ago

Collaborator

codigo fuente creado para el feature 6 con validaciones y pruebas



Mention @copilot in a comment to make changes to this pull request.

GiovanniSambonino added 2 commits 18 minutes ago

feat: Registrar un libro en el catálogo, con un título y un código ún...

2351dba

fix: Removiendo comentarios

fc679f4



lacedeno11 commented 1 minute ago

Collaborator

todo correcto



Review required

At least 1 approving review is required by reviewers with write access.

All checks have passed

1 successful check

Merging is blocked

At least 1 approving review is required by reviewers with write access.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Reviewers

Copilot

Request

Suggestions

Darvier88

Request

At least 1 approving review is required to merge this pull request.

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Unsubscribe

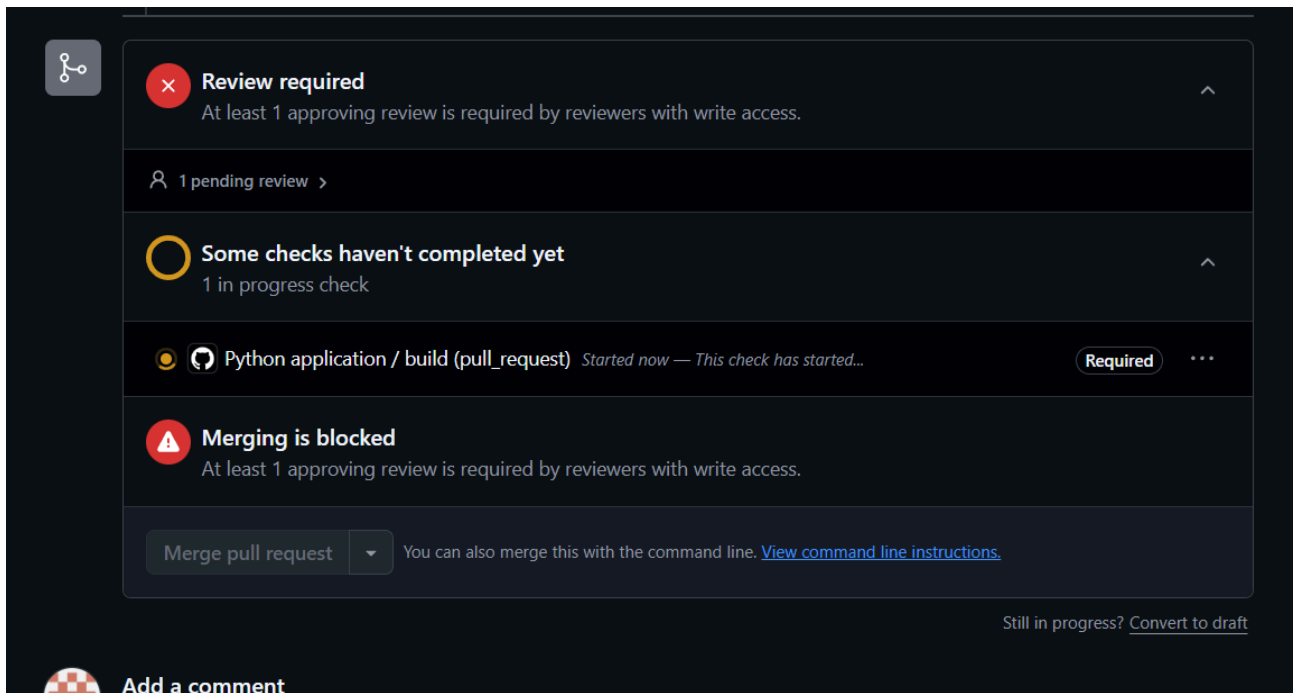
You're receiving notifications because you authored the thread.

1 participant



Lock conversation

Jonathan Zambrano



Review required
At least 1 approving review is required by reviewers with write access.

1 pending review

Some checks haven't completed yet
1 in progress check

Python application / build (pull_request) Started now — This check has started... **Required**

Merging is blocked
At least 1 approving review is required by reviewers with write access.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Add a comment

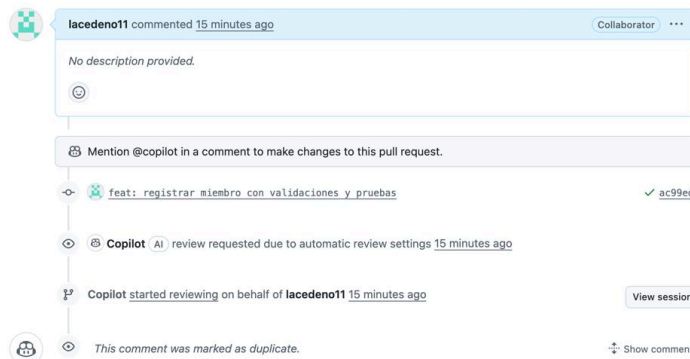
Luis Cedeño

feat: registrar miembro con validaciones y pruebas #1

Awaiting approval Code

Open lacedeno11 wants to merge 1 commit into main from luis_cedeno_feature1

Conversation 3 Commits 1 Checks 1 Files changed 5 +167



lacedeno11 commented 15 minutes ago

No description provided.

Mention @copilot in a comment to make changes to this pull request.

feat: registrar miembro con validaciones y pruebas

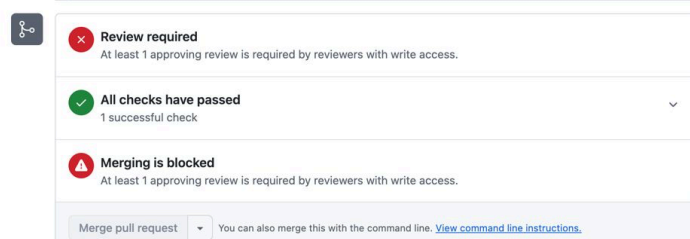
Copilot AI review requested due to automatic review settings 15 minutes ago

Copilot started reviewing on behalf of lacedeno11 15 minutes ago

This comment was marked as duplicate.

Collaborator

Show comment

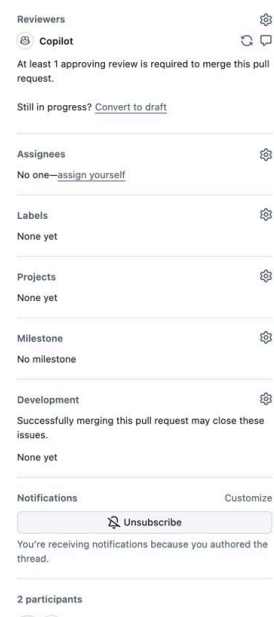


Review required
At least 1 approving review is required by reviewers with write access.

All checks have passed
1 successful check

Merging is blocked
At least 1 approving review is required by reviewers with write access.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)



Reviewers

Copilot

At least 1 approving review is required to merge this pull request.

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

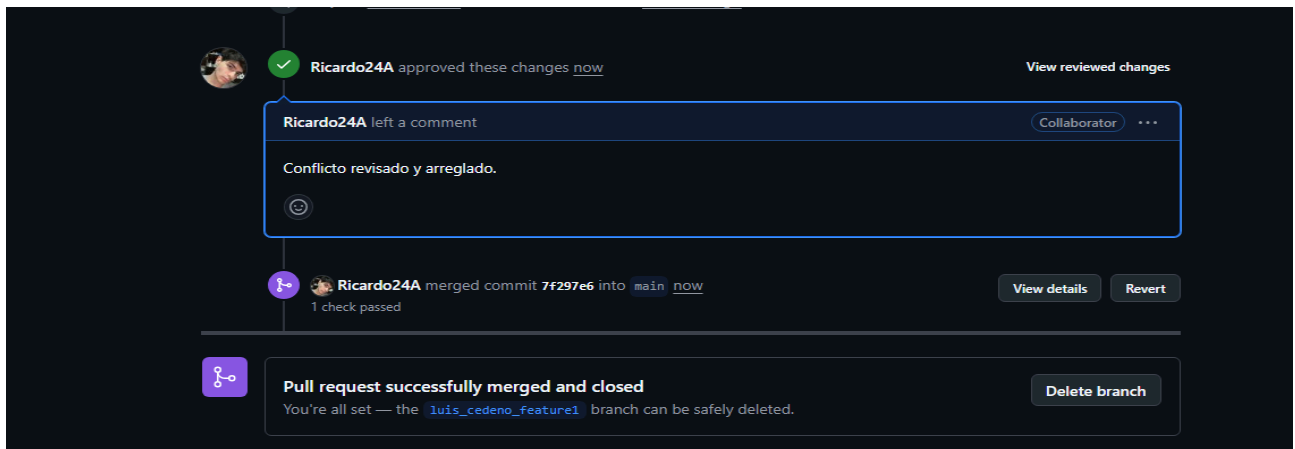
[Unsubscribe](#) Customize

You're receiving notifications because you authored the thread.

2 participants

Review captures (approving review per member)

Ricardo Asanza



This screenshot shows a GitHub pull request interface for Ricardo Asanza. At the top, a green checkmark indicates that Ricardo24A approved the changes. Below this, a comment box shows Ricardo24A's comment: "Conflicto revisado y arreglado." with a smiley face emoji. Further down, a merge event is shown: Ricardo24A merged commit 7f297e6 into main, with 1 check passed. At the bottom, a purple box states "Pull request successfully merged and closed" and provides instructions on deleting the luis_cedeno_feature1 branch.

  Ricardo24A approved these changes [now](#) [View reviewed changes](#)

Ricardo24A left a comment [Collaborator](#) [...](#)

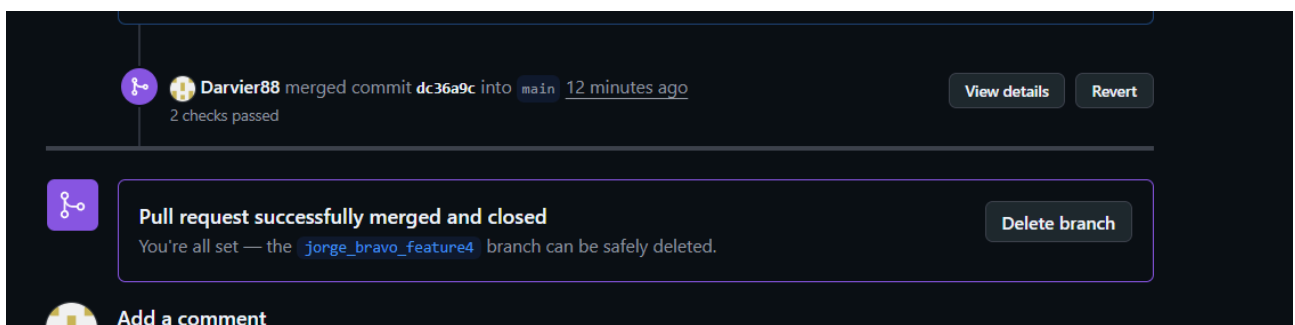
Conflicto revisado y arreglado.



  Ricardo24A merged commit **7f297e6** into `main` [now](#)
1 check passed [View details](#) [Revert](#)

 **Pull request successfully merged and closed** [Delete branch](#)
You're all set — the `luis_cedeno_feature1` branch can be safely deleted.

Darwin Pacheco



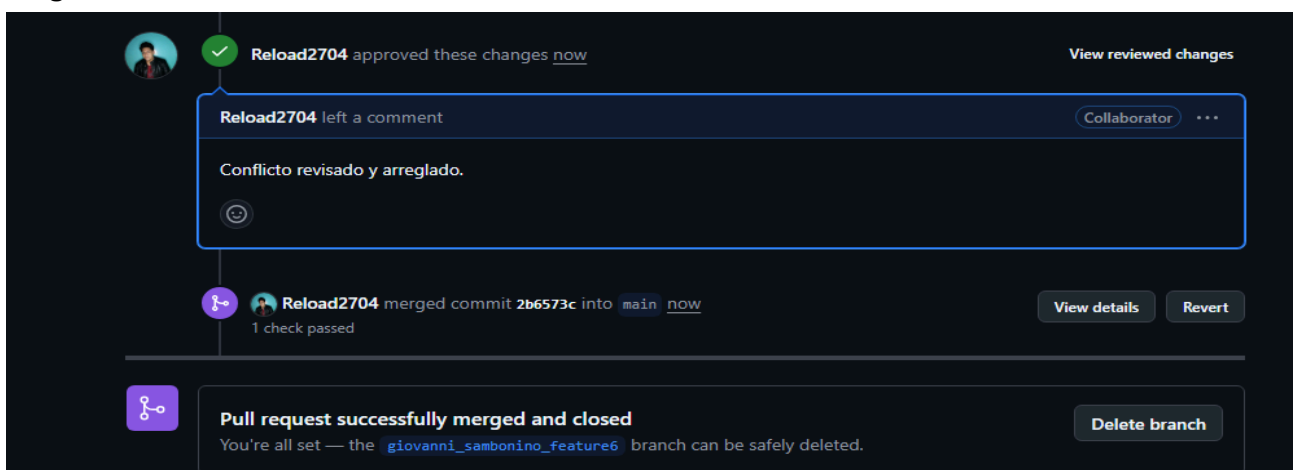
This screenshot shows a GitHub pull request interface for Darwin Pacheco. It features a merge event where Darvier88 merged commit dc36a9c into main, 12 minutes ago, with 2 checks passed. Below the merge, a purple box indicates the pull request was successfully merged and closed, suggesting the jorge_bravo_feature4 branch can be safely deleted. At the bottom, there is an "Add a comment" button.

  Darvier88 merged commit **dc36a9c** into `main` [12 minutes ago](#)
2 checks passed [View details](#) [Revert](#)

 **Pull request successfully merged and closed** [Delete branch](#)
You're all set — the `jorge_bravo_feature4` branch can be safely deleted.

 [Add a comment](#)

Jorge Bravo



This screenshot shows a GitHub pull request interface for Jorge Bravo. It displays an approval from Reload2704, a comment from Reload2704 stating "Conflicto revisado y arreglado." with a smiley face emoji, and a merge event where Reload2704 merged commit 2b6573c into main, with 1 check passed. At the bottom, a purple box confirms the pull request was successfully merged and closed, advising that the giovanni_sambonino_feature6 branch can be safely deleted.

  Reload2704 approved these changes [now](#) [View reviewed changes](#)

Reload2704 left a comment [Collaborator](#) [...](#)

Conflicto revisado y arreglado.



  Reload2704 merged commit **2b6573c** into `main` [now](#)
1 check passed [View details](#) [Revert](#)

 **Pull request successfully merged and closed** [Delete branch](#)
You're all set — the `giovanni_sambonino_feature6` branch can be safely deleted.

Giovanni Sambonino

This screenshot shows a GitHub pull request interface for user Giovanni Sambonino. At the top, a purple 'Merged' badge is visible. The title of the pull request is 'feat: add loan command with business rules and tests #6', and it indicates that GiovanniSambonino merged 4 commits into the 'main' branch from the 'jonathan_zambrano_feature2' branch. The commit history shows 'Copilot finished work on behalf of GiovanniSambonino' 5 minutes ago, followed by 'Copilot (AI) requested a review from GiovanniSambonino' 5 minutes ago. A green checkmark indicates that GiovanniSambonino approved these changes 1 minute ago. A comment box is open, showing a comment from GiovanniSambonino: 'Correcciones realizadas'. Below the comment, it shows that GiovanniSambonino merged commit '41fba91' into 'main' now, with 1 check passed. At the bottom, a summary box states 'Pull request successfully merged and closed' and provides instructions on deleting the 'jonathan_zambrano_feature2' branch.

Merged feat: add loan command with business rules and tests #6
GiovanniSambonino merged 4 commits into `main` from `jonathan_zambrano_feature2`

Copilot finished work on behalf of GiovanniSambonino 5 minutes ago

Copilot (AI) requested a review from GiovanniSambonino 5 minutes ago

GiovanniSambonino approved these changes 1 minute ago [View reviewed changes](#)

GiovanniSambonino left a comment [Collaborator](#) ...

Correcciones realizadas

GiovanniSambonino merged commit 41fba91 into `main` now
1 check passed [View details](#) [Revert](#)

Pull request successfully merged and closed
You're all set — the `jonathan_zambrano_feature2` branch can be safely deleted. [Delete branch](#)

Jonathan Zambrano

This screenshot shows a GitHub pull request interface for user Jonathan Zambrano. A purple 'Merged' badge is at the top. The title is 'Feature 5 completed #5', indicating that Jonthz merged 5 commits into the 'main' branch from the 'darwin_pacheco_feature5' branch. The commit history shows 'Merge branch 'main' into darwin_pacheco_feature5' with a 'Verified' status and commit hash 'dc904ab'. A green checkmark shows that Jonthz approved these changes 29 minutes ago. Below this, it shows Jonthz merged commit 'a287f65' into 'main' 28 minutes ago with 1 check passed. A comment from GiovanniSambonino is visible, asking '@copilot resolve the merge conflicts in this pull request'. At the bottom, a summary box states 'Pull request successfully merged and closed' and provides instructions on deleting the 'darwin_pacheco_feature5' branch.

Merged Feature 5 completed #5
Jonthz merged 5 commits into `main` from `darwin_pacheco_feature5`

GiovanniSambonino commented 8 hours ago [Collaborator](#) ...

@copilot resolve the merge conflicts in this pull request

Merge branch 'main' into darwin_pacheco_feature5 [Verified](#) ✓ dc904ab

Jonthz approved these changes 29 minutes ago [View reviewed changes](#)

Jonthz merged commit a287f65 into `main` 28 minutes ago
1 check passed [View details](#) [Revert](#)

Pull request successfully merged and closed
You're all set — the `darwin_pacheco_feature5` branch can be safely deleted. [Delete branch](#)

Luis Cedeño

This screenshot shows a GitHub pull request interface for user Luis Cedeño. A green checkmark indicates that lacedeno11 approved these changes 17 hours ago. Below this, it shows lacedeno11 merged commit '741cb0e' into 'main' 17 hours ago with 2 checks passed. At the bottom, a summary box states 'Pull request successfully merged and closed' and provides instructions on deleting the 'ricardo_asanza_feature3' branch.

lacedeno11 approved these changes 17 hours ago [View reviewed changes](#)

lacedeno11 merged commit 741cb0e into `main` 17 hours ago
2 checks passed [View details](#) [Revert](#)

Pull request successfully merged and closed
You're all set — the `ricardo_asanza_feature3` branch can be safely deleted. [Delete branch](#)

2.5 The CI blocks a change that breaks the tests

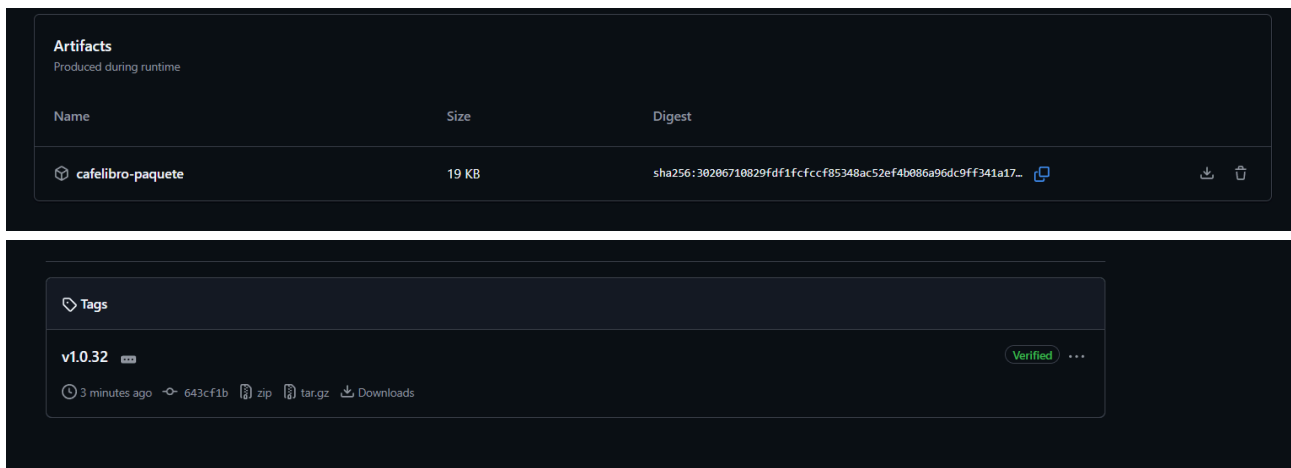
On a branch, a change that breaks an existing test was introduced; the PR check went red and blocked the merge. In the next commit the change was fixed and the check went green again, unblocking the merge.

The screenshot shows a GitHub Actions workflow for a Python application. The workflow is named "feat: Listar los libros de un miembro que ha prestado #8". The status is "Failure". The workflow file is "python-app.yml" on the "pull_request" event. The build step failed with an exit code of 5. The annotations show 1 error and 1 warning. The error message is: "Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/s".

The screenshot shows a GitHub Actions workflow for a Python application. The workflow is named "feat: Listar los libros de un miembro que ha prestado #11". The status is "Success". The workflow file is "python-app.yml" on the "pull_request" event. The build step succeeded. The annotations show 1 warning. The build log shows the following steps: Set up job (1s), Run actions/checkout@v4 (1s), Set up Python 3.10 (0s), Install dependencies (4s), Lint with flake8 (0s), Test with pytest (0s), Post Set up Python 3.10 (1s), Post Run actions/checkout@v4 (0s), and Complete job (0s).

2.6 Automating the release (Activity 6)

When a PR is merged into main with the build green, the pipeline packages the application (e.g. a Python wheel / npm tarball / JAR), attaches it as a downloadable artifact of the run, and creates a version tag for that commit.



2.7 Decision on Continuous Deployment (Activity 7)

Argue the decision for the team's own pipeline. Answer explicitly:

1. What you would have to change in the workflow to make releases happen automatically, with no one approving them.
2. If a bug got past the tests, it would reach users with no one checking first: which features would be risky to release this way, and why?
3. Research at least one of these safeguards and cite the source: a way to undo a bad release (rollback), a copy of the system to test on first (staging), or alarms when something fails in production (monitoring/alerting).
4. Final decision: would you turn on continuous deployment now? What would have to be true for you to change your mind?

The team decided not to enable continuous deployment for CaféLibro at this stage. To activate it, the release job in the workflow would need to run unconditionally on every merge to main, removing the manual approval gate and deploying the packaged artifact automatically [3]. The riskiest feature to release without human review would be the loan function (Feature 2): a silent bug in the duplicate-loan check could allow a book to appear on loan to two members simultaneously, corrupting the shared JSON state file. The return function (Feature 3) carries similar risk because an incorrect availability flag could make a book permanently unloanable. Regarding safeguards, the team researched rollback via Git tags: because the pipeline already creates a version tag on each passing merge, reverting a bad release requires a single revert command that restores the exact source state that was previously known to pass [3]. The team would reconsider enabling continuous deployment once the test suite covers all business-rule edge cases, a staging environment exists to run smoke tests before production, and an alerting mechanism notifies the team when a deployed version produces errors.

3. Questions

The following questions are answered in the team's own words, drawing on the experience of this practice.

3.1 Why is CI worth the effort on a team project? What goes wrong without it?

CI is worth the setup effort because defects on a shared repository compound quickly. Without it, each developer integrates their changes only when they believe the feature is complete, which means two developers can spend a week on parallel branches and discover a conflict only at the end. By that point, both changes are entangled and the conflict is expensive to untangle. With CI, each push triggers a build on a clean machine: if the build breaks, only one recent change is the culprit, and fixing it takes

minutes rather than hours. The practice also surfaces test failures before they reach the shared branch, so the main branch stays in a releasable state at all times. On the CaféLibro project specifically, having five features touch the same JSON state file made late integration particularly risky; CI caught the cases where one feature's changes to the data schema broke another member's tests.

3.2 Comparison with at least two other CI/CD tools (Jenkins, GitLab CI, CircleCI, Travis CI, etc.)

Below is a comparison of two additional CI tools alongside the GitHub Actions setup used in this project.

Jenkins is a self-hosted, open-source automation server widely used in enterprise environments [4]. A project configures its pipeline in a Jenkinsfile stored in the repository root, written in a Groovy-based DSL. Builds run on the Jenkins server or on agents the team provisions, so the team is responsible for installing and maintaining the infrastructure. Jenkins has a large plugin ecosystem and is highly flexible, but initial setup takes considerably more time than hosted alternatives. GitLab CI is integrated directly into the GitLab platform and configured via a .gitlab-ci.yml file at the repository root, using YAML syntax similar to GitHub Actions [5]. Builds run on GitLab-managed runners at no cost on gitlab.com up to a monthly minute limit, or on self-hosted runners registered by the team. Because source code, issue tracker, and pipeline all live in the same platform, no external service connections are needed, which reduces setup friction compared with Jenkins. GitHub Actions, used in this project, is also configured via a YAML file (.github/workflows/ci.yml) and runs on GitHub-hosted runners at no cost for public repositories [1].

3.3 Which one would you choose for the course project, and why?

The team evaluated GitHub Actions, Jenkins, and GitLab CI on three criteria: code location, cost, and setup effort.

For this course project the team would choose GitHub Actions [1]. The repository is already hosted on GitHub, so there is no need to connect an external service or create accounts on another platform. The workflow file requires no installed infrastructure: GitHub provides the runners at no cost for public repositories and at a generous free tier for private ones. Jenkins would offer more control but demands a server to run and maintain, which is unnecessary overhead for a five-person academic project. GitLab CI is a reasonable alternative but would require migrating the repository to GitLab or configuring a mirror, adding complexity without a clear benefit given the team is already on GitHub.

3.4 How does CI fit with the way agile teams work in short iterations?

Agile teams work in short iterations — typically one to two weeks — and aim to have a shippable increment at the end of each one. CI supports this directly: because every merge is tested automatically, the team always knows whether the current state of main is releasable. There is no “integration sprint” at the end; integration happens continuously throughout the iteration. CI also reinforces the agile practice of small, frequent commits: developers are motivated to push small changes because large, long-lived branches accumulate merge conflicts and test failures that are harder to resolve. In the CaféLibro project, the round-review flow — where each member reviewed the pull request of the next member — mirrors the agile principle of team ownership and collective code review, and the pipeline provided immediate feedback on whether each increment was ready to merge.

4. Conclusions and recommendations

- The team successfully built and validated a complete CI pipeline: a shared repository with branch protections, a GitHub Actions workflow that runs pytest on every push and pull request, a release step that packages the application and creates a version tag, and a round-review process in which each member contributed code and reviewed a colleague's pull request. The main branch was kept passing throughout the exercise.
- Working through the branch-and-pull-request flow revealed how much the quality of a shared codebase depends on small, incremental merges combined with automated testing.

When a deliberately broken commit was pushed, the CI check blocked the merge within seconds — a concrete demonstration that the pipeline does exactly what it promises: protect the main branch from regressions.

- Teams starting with CI should invest time upfront in writing meaningful tests, not just tests that pass trivially. The pipeline is only as useful as the test suite it runs; a green check on a weak suite gives false confidence.
- Before enabling continuous deployment, teams should add a staging environment and at least one smoke test that exercises the most critical user path end-to-end. For CaféLibro, that would mean verifying that a member can be registered, borrow a book, and return it in a single automated run against a clean data file.

5. References

Suggested format: IEEE or APA. Examples:

[1] GitHub, “About continuous integration,” GitHub Docs, docs.github.com/en/actions/automating-builds-and-tests/about-continuous-integration, 2024.

[2] M. Fowler, “Continuous Integration,” [martinfowler.com](https://martinfowler.com/articles/continuousIntegration.html), <https://martinfowler.com/articles/continuousIntegration.html>, 2006 (updated 2023).

[3] GitHub, “Managing releases in a repository,” GitHub Docs, docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository, 2024.